

Documentación Técnica del Proyecto SumixKids

1. Estructura del Proyecto

El proyecto está organizado siguiendo una arquitectura estándar de aplicaciones Java web con Maven. La estructura principal es la siguiente:

- **src/main/java/com/sumixkids/**
 - **config/**: Clases de configuración y utilidades de arranque y base de datos.
 - BootstrapListener.java: Inicializa roles y configura la zona horaria al arrancar la aplicación.
 - DatabaseManager.java: Gestiona la conexión a la base de datos MySQL.
 - **dao/**: Acceso a datos (Data Access Objects) para roles, usuarios y otros recursos.
 - UsuarioDAO.java: Métodos CRUD para usuarios, validación de login, registro, bloqueo, etc.
 - RoleDAO.java: Métodos para consultar y gestionar roles.
 - PasswordResetDAO.java, TwoFactorCodeDAO.java, CodigoRecuperacionDAO.java: Manejo de recuperación de contraseñas y códigos 2FA.
 - LogAuditoriaDAO.java: Gestión de logs de auditoría y seguimiento
 - **filter/**:
 - AdminSecurityFilter.java: Control de acceso a rutas administrativas
 - SessionTimeoutFilter.java: Manejo de timeout de sesiones
 - **model/**: Modelos de datos.
 - Usuario.java: Representa la entidad usuario con sus atributos.
 - RoleType.java: Enum para los diferentes tipos de roles.
 - LogAuditoria.java: Entidad para registro de auditoría
 - ValidacionCargaMasiva.java: Modelo para validación de datos masivos
 - **service/**: Servicios de lógica de negocio.
 - EmailService.java: Envío de correos electrónicos (bienvenida, 2FA, alertas).

- **util/**: Utilidades generales.
 - PasswordUtil.java: Encriptación y verificación de contraseñas con BCrypt.
 - TwoFactorUtil.java: Generación de códigos para autenticación de dos factores.
 - ValidacionUtil.java: Utilidades de validación de datos
- **web/**: Servlets para la lógica de presentación y control de flujo web.
 - LoginServlet.java: Maneja el inicio de sesión, validación de usuario, bloqueo tras intentos fallidos, envío de 2FA.
 - RegistroServlet.java: Maneja el registro de nuevos usuarios, validaciones y envío de correo de bienvenida.
 - BienvenidaServlet.java: Muestra la pantalla de bienvenida tras login exitoso.
 - LogoutServlet.java: Cierra la sesión del usuario.
 - RecuperarServlet.java, RestablecerServlet.java: Recuperación y restablecimiento de contraseñas.
 - CambiarRolUsuarioServlet.java: Permite a un administrador cambiar el rol de un usuario.
 - EliminarUsuarioServlet.java: Permite eliminar usuarios, validando que no tengan registros asociados.
 - SecurityFilter.java: Filtro global que restringe el acceso a rutas según el rol del usuario.
 - TwoFactorServlet.java: Maneja la verificación del código 2FA.
 - AdminUsuarioServlet.java: Gestión administrativa de usuarios
 - AuditoriaServlet.java: Visualización de logs de auditoría
 - CargaMasivaServlet.java: Manejo de cargas masivas
- **src/main/resources/**
 - **config.properties**: Configuración de la aplicación (parámetros de seguridad, correo, etc.).
 - **logback.xml**: Configuración de logging.

- **src/main/webapp/**
 - **JS:**
 - session-timeout.js: Control de timeout en el cliente
 - **JSPs:**
 - index.jsp: Página de inicio.
 - login.jsp: Formulario de inicio de sesión.
 - registro.jsp: Formulario de registro de usuario.
 - bienvenida.jsp: Pantalla de bienvenida tras login.
 - recuperar.jsp, restablecer.jsp: Formularios para recuperación y restablecimiento de contraseña.
 - 2fa.jsp: Pantalla para ingresar el código de autenticación de dos factores.
 - auditoria.jsp: Vista de logs de auditoría
 - carga_masiva.jsp: Interfaz de carga masiva
 - eliminar_confirmar.jsp: Confirmación de eliminación
 - timeout.jsp: Página de sesión expirada
 - usuario.jsp: Gestión de usuarios
 - **Recursos estáticos:**
 - css/styles.css: Estilos personalizados.
 - images/sumixkids.png: Logo de la aplicación.
 - **WEB-INF/**
 - web.xml: Configuración principal de la aplicación web (servlets, filtros, recursos).
 - **META-INF/**
 - context.xml: Configuración de recursos JNDI para la base de datos.
- **pom.xml:** Archivo de configuración de Maven (dependencias, plugins, build).
- **sumixkids.sql:** Script de creación y estructura de la base de datos.
- **docs/:** Documentación adicional del proyecto.

2. Funcionalidades Implementadas

- **Registro y Login de Usuarios:**

- Implementados mediante RegistroServlet y LoginServlet junto con sus respectivas páginas JSP.
- Validación de campos obligatorios, formato de correo, unicidad de usuario/correo y políticas de contraseña.
- Contraseñas almacenadas de forma segura usando BCrypt.
- Bloqueo automático de cuenta tras 3 intentos fallidos de login y notificación por correo.
- Autenticación de dos factores (2FA) por correo electrónico tras login exitoso.

- **Validación:**

- Validación de formatos de datos
- Sistema de validación para cargas masivas
- Confirmación para eliminación de usuarios

- **Gestión de Roles:**

- Definición y consulta de roles mediante RoleType y RoleDAO.
- Solo administradores pueden cambiar roles (CambiarRolUsuarioServlet).

- **Gestión de Sesiones:**

- Control de timeout automático
- Redirección a página de timeout
- Protección de rutas administrativas

- **Bienvenida y Logout:**

- BienvenidaServlet muestra la pantalla de bienvenida tras login.
- LogoutServlet cierra la sesión y redirige al login.

- **Recuperación y Restablecimiento de Contraseña:**

- RecuperarServlet y RestablecerServlet permiten solicitar y cambiar la contraseña mediante correo.

- **Eliminación de Usuarios:**

- EliminarUsuarioServlet permite eliminar usuarios solo si no tienen registros asociados en tablas críticas.

- **Seguridad y Restricción de Acceso:**
 - SecurityFilter restringe el acceso a rutas según el rol del usuario (principio de mínimo privilegio).
 - Validación de sesión activa y protección de rutas administrativas.
- **Envío de Correos:**
 - EmailService gestiona el envío de correos para bienvenida, alertas de seguridad, 2FA y recuperación de contraseña.
- **Auditoría y Logging:**
 - Registro detallado de acciones de usuarios
 - Visualización de logs con filtros
 - Trazabilidad de cambios en el sistema
- **Persistencia y Conexión a Base de Datos:**
 - DatabaseManager gestiona la conexión a MySQL.
 - DAOs implementan operaciones CRUD y validaciones.
- **Utilidades de Seguridad:**
 - PasswordUtil para encriptación y verificación de contraseñas.
 - TwoFactorUtil para generación de códigos 2FA.

3. Dependencias Utilizadas

- jbcrypt: Para encriptación segura de contraseñas (hashing con sal y múltiples rondas).
- mysql-connector-j: Driver JDBC para conexión a base de datos MySQL.
- jstl: Soporte para JSTL en páginas JSP (etiquetas de lógica, iteración, etc.).
- logback-classic y logback-core: Logging flexible y configurable.
- slf4j-api: API de logging utilizada por logback.
- jakarta.mail y jakarta.activation: Envío de correos electrónicos.
- protobuf-java: Incluida, aunque actualmente no se utiliza en la lógica principal.
- Otros jars: Bibliotecas de soporte para el servidor y la aplicación web.

4. Seguridad

- Contraseñas: Nunca se almacenan en texto plano, siempre cifradas con BCrypt.
- Políticas de Contraseña: Mínimo 8 caracteres, mayúsculas, minúsculas, números y caracteres especiales.

- Bloqueo de Cuenta: Tras 3 intentos fallidos, la cuenta se bloquea y se notifica al usuario.
- 2FA: Autenticación de dos factores por correo electrónico.
- Restricción de Acceso: Filtro global que protege rutas y funciones según el rol.
- Validación de Datos: Todos los formularios validan campos obligatorios y formatos.
- Sesión: (Pendiente) No está configurado el cierre automático por inactividad, pero puede añadirse fácilmente.

5. Estado Actual

- El proyecto compila y genera un archivo WAR (sumixkids.war).
- Páginas JSP funcionales para registro, login, bienvenida y recuperación de contraseña.
- Base de datos definida y lista para uso con el script sumixkids.sql.
- Código modular y preparado para ampliaciones futuras (más servicios, DAOs, etc.).
- Logging configurado para registrar eventos y errores.

6. Pendientes y Próximos Pasos

- Implementar pruebas unitarias y de integración.
 - Mejorar la interfaz de usuario (JSP/CSS, experiencia de usuario).
 - Añadir cierre automático de sesión por inactividad.
 - Implementar modo mantenimiento y avisos programados.
 - Documentar el código y los procesos de despliegue.
 - Optimizar rendimiento y tiempos de carga.
 - Configurar HTTPS/TLS en el servidor para cifrar las comunicaciones.
 - Añadir más funcionalidades según los requerimientos del proyecto.
-

Historias de Usuario

1. Seguridad en Contraseñas

- **HU-01:** Como administrador, quiero que el sistema verifique que la contraseña cumpla con las políticas de seguridad (mínimo 8 caracteres, mayúsculas, minúsculas, números y caracteres especiales), para evitar accesos no autorizados. ✓
 - **HU-02:** Como administrador, quiero que el sistema bloquee el acceso tras tres intentos fallidos de inicio de sesión y envíe una alerta al correo registrado, para proteger las cuentas contra intentos de fuerza bruta. ✓
 - **HU-03:** Como administrador, quiero que el inicio de sesión requiera autenticación de dos factores (2FA). ✓
-

2. Creación de Usuarios (Create)

- **HU-04:** Como administrador, quiero que al registrar un usuario nuevo se verifique que el correo electrónico no esté ya registrado, para evitar duplicados en la base de datos. ✓
 - **HU-05:** Como administrador, quiero que la asignación de roles solo se pueda realizar a usuarios autorizados, para asegurar que solo las personas correctas tengan privilegios especiales. ✓
-

3. Eliminación de Usuarios (Delete)

- **HU-06:** Como administrador, quiero que al eliminar un usuario se compruebe que no tenga procesos o registros activos asociados, para no generar inconsistencias o pérdida de información relacionada. ✓
 - **HU-10:** Como administrador, quiero que cualquier eliminación de datos importantes solicite confirmación y requiera reautenticación, para prevenir borrados accidentales o malintencionados. ✓
-

4. Validaciones Generales de Datos

- **HU-07:** Como administrador, quiero que todos los campos obligatorios estén claramente identificados y no permitan guardar si están vacíos, para garantizar la integridad de la información. ✓
 - **HU-08:** Como administrador, quiero que el sistema valide formatos de datos (correos electrónicos, fechas, números), para evitar errores y datos inválidos. ✓
 - **HU-09:** Como administrador, quiero que el sistema muestre mensajes de error claros y específicos cuando la validación falle, para que el usuario sepa exactamente cómo corregir la información. ✓
-

5. Actualización y Registro de Cambios (Update)

- **HU-11:** Como administrador, quiero que todos los cambios de configuración queden registrados en un log seguro con fecha, hora y usuario responsable, para mantener un historial de auditoría. ✓
 - **HU-13:** Como administrador, quiero que el sistema restrinja el acceso a funciones y módulos de acuerdo con el rol del usuario, para aplicar el principio de mínimo privilegio. ✓
 - **HU-15:** Como administrador, quiero que cualquier cambio en los roles de un usuario requiera doble validación (otro administrador o superusuario), para evitar cambios indebidos en los permisos. +
-

6. Gestión de Cargas Masivas

- **HU-12:** Como administrador, quiero que las cargas masivas de datos sean validadas en formato y coherencia antes de guardarse, para prevenir registros erróneos o corruptos. ✓
-

7. Seguridad y Control de Accesos

- **HU-14:** Como administrador, quiero que todos los intentos de acceso no autorizados queden registrados con fecha, hora y dirección IP, para facilitar la detección de amenazas. +
- **HU-18:** Como administrador, quiero que todas las comunicaciones entre el cliente y el servidor estén cifradas mediante HTTPS/TLS, para proteger la información en tránsito. +

- **HU-19:** Como administrador, quiero que el sistema cierre la sesión automáticamente tras 10 minutos de inactividad, para evitar accesos no autorizados en equipos desatendidos. ✓
 - **HU-20:** Como administrador, quiero que las contraseñas se almacenen cifradas con algoritmos seguros, para protegerlas en caso de filtraciones de datos. ✓
-

8. Rendimiento y Mantenimiento

- **HU-16:** Como administrador, quiero que el panel cargue en menos de 3 segundos en condiciones normales, para ofrecer una buena experiencia de usuario. **NO HAY CODIGO ESPECIFICO QUE ASEGURE ESTO, YA QUE DEPENDE DEL SERVIDOR, CONEXIÓN A INTERNET, BASE DE DATOS Y CANTIDAD DE DATOS... ESTA NO LA VAMOS A HACER**
- **HU-17:** Como administrador, quiero que el sistema muestre un aviso de mantenimiento programado y limite el uso de ciertas funciones durante ese tiempo, para evitar errores en la información. +

Actualizado El 19/09/2025